



南開大學
Nankai University

计算机学院
软件工程实验报告

实验 4: 软件编程实现、分析和测试

姓名：王旭

学号：2312166

专业：计算机科学与技术

2026 年 6 月 4 日

目录

1 问题描述及分析	4
1.1 问题描述	4
1.1.1 题目背景	4
1.1.2 形式化定义	4
1.1.3 输入与输出约定	4
1.1.4 题目示例	4
1.2 问题分析	5
1.2.1 朴素思路：暴力枚举	5
1.2.2 优化思路：单调双端队列	6
1.2.3 两种算法对比	7
1.2.4 本实验的技术路线	7
2 原始代码	7
2.1 代码文件说明	7
2.2 类结构与职责	8
2.3 完整源代码	8
2.4 核心逻辑说明	9
2.5 运行示例	9
2.6 原始实现的不足	9
2.7 与后续章节的关系	10
3 Pylint 代码分析	10
3.1 检查命令与配置	10
3.1.1 采用的规范	10
3.1.2 Pylint 配置文件	10
3.1.3 检查命令	11
3.2 首次检查结果与问题修复	11
3.2.1 首次检查概况	11
3.2.2 问题清单与修复措施	12
3.2.3 典型问题说明	12
3.3 修复后结果	12
3.3.1 基础检查：满分	12
3.3.2 详细报告 (-ry) 统计摘要	13
3.4 主要代码修改对照	14
4 Profile 性能分析	15
4.1 分析方法	15
4.1.1 性能分析工具	15
4.1.2 测试数据生成	16
4.1.3 实验命令	16
4.1.4 复杂度理论基准	16

4.2	墙钟时间对比	17
4.2.1	测试环境与方法	17
4.2.2	实测结果	17
4.2.3	结果分析	17
4.3	Profile 结果解读	17
4.3.1	朴素算法 Profile ($\text{scale} = 30000, k = 3000$)	17
4.3.2	优化算法 Profile ($\text{scale} = 500000, k = 1000$)	18
4.4	优化总结	19
4.4.1	优化前后对照	19
4.4.2	优化思路回顾	19
4.5	本章小结与作业问题 e) 对应	20
5	Unittest 单元测试	20
5.1	测试设计思路	20
5.1.1	测试策略：黑盒与白盒相结合	20
5.1.2	黑盒测试设计维度	21
5.1.3	白盒测试设计维度	21
5.1.4	测试类结构	22
5.2	测试用例表	22
5.3	测试代码说明	22
5.3.1	测试文件与运行方式	22
5.3.2	代表性测试代码	23
5.3.3	测试与工程实践的对应	23
5.4	测试运行结果	24
5.4.1	执行结果	24
5.4.2	结果汇总	24
5.5	本章小结与作业问题 f) 对应（前半）	25
6	代码覆盖率	25
6.1	覆盖率工具与命令	25
6.1.1	选用的覆盖指标	25
6.1.2	工具与配置文件	25
6.1.3	运行命令	25
6.2	覆盖率结果	26
6.2.1	执行过程	26
6.2.2	终端覆盖率摘要	27
6.2.3	HTML 覆盖率报告总览	27
6.3	未覆盖代码说明	28
6.4	缺陷报告	30
6.4.1	被测程序缺陷	30
6.4.2	测试与用例相关缺陷	30
6.5	本章小结与作业问题 f) 对应	30

7 编程规范	30
7.1 是否遵守编程规范?	31
7.2 参考的规范	31
7.3 如何检查是否遵守编程规范?	31
7.4 小结: 作业问题 a) 一览	32
8 程序可扩展性	32
8.1 已实现的扩展	32
8.2 未来可扩展方向	33
8.3 小结: 作业问题 b) 一览	34
9 两个原则	34
9.1 单一职责原则 (SRP)	34
9.1.1 原则含义	34
9.1.2 本实验中的体现	34
9.1.3 独立原始模块	35
9.2 开放-封闭原则 (OCP)	35
9.2.1 原则含义	35
9.2.2 本实验中的体现	35
9.2.3 与第八章的关系	36
9.3 可复用且稳定的基础功能	36
9.4 小结: 作业问题 c) 一览	36
10 错误处理	36
10.1 输入约定 (前置条件)	37
10.2 已考虑的错误情形	37
10.3 处理机制说明	37
10.4 小结: 作业问题 d) 一览	38
11 完整程序代码	38
11.1 源代码文件清单	38
11.1.1 核心源代码	38
11.1.2 辅助脚本与配置	39
11.2 实验复现命令汇总	39
12 github 仓库	39

1 问题描述及分析

1.1 问题描述

1.1.1 题目背景

滑动窗口是算法与软件工程中一类常见的问题模型。在实际系统中，许多场景都需要在**固定长度的连续数据段**上快速获取统计特征，例如：网络流量监控中统计最近 k 个时间点的峰值带宽；股票行情分析中计算最近 k 个交易日内的最高价格；传感器数据流处理中检测滑动区间内的异常峰值等。若每次窗口右移都重新扫描整个窗口，在数据规模较大时将带来不可接受的性能开销。因此，**高效维护滑动窗口最值**既是经典算法题，也具有明确的工程意义。

本实验题目为：**滑动窗口最大值 (Sliding Window Maximum)**。

1.1.2 形式化定义

给定一个整数数组 $nums$ ，数组长度为 n ，以及一个正整数窗口大小 k （满足 $1 \leq k \leq n$ ）。有一个长度为 k 的滑动窗口从数组 $nums$ 的最左侧移动到最右侧；每次移动时，窗口仅向右滑动一位，即每次丢弃最左端一个元素、纳入最右端下一个元素。

在任意时刻，窗口内恰好包含 k 个连续元素。请找出**每次窗口移动后**（含初始窗口）窗口内元素的最大值，并按顺序返回由这些最大值组成的数组。

记输出数组为 $result$ ，则其长度应为 $n - k + 1$ 。对于每个窗口起始下标 i ($0 \leq i \leq n - k$)，有：

$$result[i] = \max(nums[i], nums[i + 1], \dots, nums[i + k - 1])$$

1.1.3 输入与输出约定

项目	说明
输入	整数数组 $nums$ ，窗口大小 k
输出	长度为 $n - k + 1$ 的整数数组，第 i 个元素为第 i 个窗口内的最大值
数据类型	本实验规定数组元素为 int 类型， k 为 正整数
边界条件	$k = 1$ 时输出即为原数组； $k = n$ 时输出仅含一个元素（全数组最大值）

1.1.4 题目示例

我们的作业要求给出的示例如下：

- **输入：** $nums = [1, 3, -1, -3, 5, 3, 6, 7]$, $k = 3$
- **输出：** $[3, 3, 5, 5, 6, 7]$

解释：数组长度为 $n = 8$ ，窗口大小 $k = 3$ ，共产生 $8 - 3 + 1 = 6$ 个窗口。各窗口及其最大值如下表所示：

步骤	窗口起始下标	当前窗口（含 k 个元素）	窗口内最大值
1	0	[1, 3, -1]	3
2	1	[3, -1, -3]	3
3	2	[-1, -3, 5]	5
4	3	[-3, 5, 3]	5
5	4	[5, 3, 6]	6
6	5	[3, 6, 7]	7

将各步最大值依次排列，即得输出 [3, 3, 5, 5, 6, 7]。

1.2 问题分析

1.2.1 朴素思路：暴力枚举

解决滑动窗口最大值问题，最直观的方法是：对每个窗口位置，遍历窗口内全部 k 个元素，取最大值。

算法步骤：

1. 初始化结果列表 *result* 为空；
2. 对于每个窗口起始下标 i ，从 0 到 $n - k$ ：
 - 令 $window_max = nums[i]$ ；
 - 对于 j 从 $i + 1$ 到 $i + k - 1$ ，若 $nums[j] > window_max$ ，则更新 $window_max$ ；
 - 将 $window_max$ 加入 *result*；
3. 返回 *result*。

伪代码：

```
1 for i = 0 to n - k:
2     window_max = nums[i]
3     for j = i + 1 to i + k - 1:
4         if nums[j] > window_max:
5             window_max = nums[j]
6     result.append(window_max)
7 return result
```

复杂度分析：

- **时间复杂度：**外层循环 $O(n - k + 1)$ ，内层循环 $O(k)$ ，合计 $O(n \times k)$ ；最坏情况下 $k \approx n/2$ 时接近 $O(n^2)$ 。
- **空间复杂度：**不计输出数组，仅使用常数个辅助变量，为 $O(1)$ 。

缺点：

- 当 n 、 k 较大时，重复比较同一元素多次，效率低下；

- 未利用”相邻窗口高度重叠”这一结构特点；
- 作为工程实现，难以满足大数据量场景下的性能要求。

本实验在 `src/sliding_window_max_naive.py` 及主程序中的 `max_sliding_window_naive()` 方法实现了该朴素算法，作为优化前的 **Baseline 基线代码**，供后续 Pylint 分析、Profile 性能对比及单元测试一致性校验使用。

1.2.2 优化思路：单调双端队列

为将时间复杂度降低至线性，可采用 **单调双端队列 (Monotonic Deque)** 维护”可能成为窗口最大值的元素下标”。

核心思想：

- 使用双端队列 *deque* 存储**数组下标**（而非元素值），且保证队列中对应元素值**从队首到队尾单调递减**；
- 队首下标所指向的元素，即为当前窗口的最大值；
- 当新元素进入窗口时，从队尾依次弹出所有”值小于等于新元素”的下标——这些元素不可能再成为任何后续窗口的最大值（新元素更靠右且更大或相等）；
- 当队首下标已滑出当前窗口左边界（下标 $\leq i - k$ ）时，从队首弹出；
- 每个下标最多入队、出队各一次，故总操作次数为 $O(n)$ 。

算法步骤（遍历下标 i 从 0 到 $n - 1$ ）：

1. **维护单调性**：当队列非空且 $nums[deque[-1]] \leq nums[i]$ 时，弹出队尾；
2. **入队**：将下标 i 加入队尾；
3. **弹出过期元素**：若队首下标 $\leq i - k$ ，弹出队首；
4. **记录结果**：当 $i \geq k - 1$ （第一个完整窗口形成）时，将 $nums[deque[0]]$ 加入结果。

正确性直观说明：

- 队首始终是当前窗口内”最靠左的最大值候选”：比它小的元素已被队尾弹出规则淘汰；比它大但不在窗口内的已在步骤 3 移除。
- 队尾弹出规则保证：若新元素 $nums[i]$ 不小于某旧下标对应元素，则旧元素在任何包含 i 的窗口中都不可能成为最大值。

复杂度分析：

- **时间复杂度**：每个下标最多入队、出队各一次，总计 $O(n)$ ；
- **空间复杂度**：队列中最多存储 k 个下标，为 $O(k)$ 。

1.2.3 两种算法对比

对比项	朴素暴力法	单调双端队列
时间复杂度	$O(n \times k)$	$O(n)$
空间复杂度	$O(1)$	$O(k)$
适用场景	小规模数据、教学基线	大规模数据、生产环境
本实验角色	原始代码 / 对比基线	主实现

1.2.4 本实验的技术路线

对于本实验，我们采用如下编程设计、分析与测试方法：

1. **编程实现**：使用 Python 实现朴素版与优化版两套算法，封装于 `SlidingWindowMax` 类，通过 `solve(use_naive=...)` 统一调度；
2. **静态分析**：使用 `Pylint` 依据 **PEP 8** 检查代码规范；
3. **性能分析**：使用 `profile` 模块与墙钟时间对比，验证 $O(n \times k)$ 与 $O(n)$ 的实际差异；
4. **单元测试**：使用 `unittest` 进行黑盒与白盒测试，并用 `coverage` 统计语句覆盖率；
5. **设计原则与扩展性**：在后续章节讨论 SRP、OCP、错误处理及可扩展性。

我们选择”保留朴素实现 + 优化实现”两个版本，从而在 Profile 与报告 e) 中形成**可量化的优化前后对比**，并形成”原始代码 → 优化 → 性能验证”的流程。

2 原始代码

本章给出**优化前**的朴素实现。该版本采用双重循环对每个滑动窗口暴力求最大值，时间复杂度为 $O(n \times k)$ ，思路直观、代码简短，适合作为本实验的性能对比基线与 `Pylint` 静态分析对象。

2.1 代码文件说明

本实验将原始代码单独存放于：

```
1 src/sliding_window_max_naive.py
```

该文件定义类 `SlidingWindowMaxNaive`，仅包含窗口最大值求解的最小逻辑，**不包含**输入合法性校验、单调队列优化及测试辅助函数。主程序 `src/sliding_window_max.py` 中的 `max_sliding_window_naive()` 方法实现了相同算法，供 Profile 对比与 `solve(use_naive=True)` 调用；两处核心循环逻辑一致，本章以独立文件为准进行展示与说明。

2.2 类结构与职责

成员	类型	说明
nums	List[int]	待处理的整数数组
k	int	滑动窗口大小
__init__()	构造方法	初始化空数组与 $k = 0$
set_array(nums, k)	方法	设置输入数据
max_sliding_window()	方法	暴力求解，返回最大值数组或 None

原始版本的设计目标是**尽快实现正确功能**，尚未引入第一章所述的 `array_is_legal()` 校验及 `solve()` 统一入口；非法输入（如 $k \leq 0$ 或 $k > n$ ）时，`max_sliding_window()` 直接返回 `None`。

2.3 完整源代码

```

1  """优化前的朴素实现（暴力枚举每个窗口的最大值）。"""
2
3  # pylint: disable=duplicate-code
4
5  from typing import List, Optional
6
7
8  class SlidingWindowMaxNaive:
9      """滑动窗口最大值 — 朴素版本，仅用于对比基线。"""
10
11     def __init__(self) -> None:
12         self.nums: List[int] = []
13         self.k: int = 0
14
15     def set_array(self, nums: List[int], k: int) -> None:
16         """设置待处理数组与窗口大小。"""
17         self.nums = nums
18         self.k = k
19
20     def max_sliding_window(self) -> Optional[List[int]]:
21         """对每个窗口暴力求最大值，时间复杂度  $O(n*k)$ 。"""
22         n = len(self.nums)
23         if self.k <= 0 or self.k > n:
24             return None
25
26         result: List[int] = []
27         for start in range(n - self.k + 1):
28             window_max = self.nums[start]
29             for idx in range(start + 1, start + self.k):
30                 if self.nums[idx] > window_max:
31                     window_max = self.nums[idx]
32             result.append(window_max)

```

33

`return result`

2.4 核心逻辑说明

(1) 边界判断 (第 22 ~ 24 行)

若窗口大小 $k \leq 0$ 或 $k > n$, 无法形成合法窗口, 函数返回 `None`。此处仅做简单判断, 错误时不打印提示信息——完整错误处理在主程序 `SlidingWindowMax` 类中实现。

(2) 外层循环 (第 27 行)

`for start in range(n - self.k + 1)` 枚举每个窗口的起始下标 $start$, 共 $n - k + 1$ 个窗口, 与第一章形式化定义一致。

(3) 内层循环 (第 28 ~ 31 行)

- 以当前窗口首元素 `nums[start]` 作为初始最大值 `window_max`;
- 遍历窗口内其余 $k-1$ 个元素(下标 `start+1` 至 `start+k-1`), 若发现更大值则更新 `window_max`;
- 内层循环结束即得到该窗口最大值。

(4) 结果收集 (第 32 ~ 33 行)

将每个窗口的 `window_max` 追加到 `result` 列表, 全部窗口处理完毕后返回。

2.5 运行示例

以题目示例为例, 我们的调用方式如下:

```
1 from sliding_window_max_naive import SlidingWindowMaxNaive
2
3 solver = SlidingWindowMaxNaive()
4 solver.set_array([1, 3, -1, -3, 5, 3, 6, 7], k=3)
5 print(solver.max_sliding_window()) # 输出: [3, 3, 5, 5, 6, 7]
```

第一个窗口 (`start=0`): 窗口 `[1, 3, -1]`, 内层比较后 `window_max=3`; 第二个窗口 (`start=1`): 窗口 `[3, -1, -3]`, `window_max=3`; 依此类推, 最终得到 `[3, 3, 5, 5, 6, 7]`, 与优化算法输出一致 (第五章将用单元测试自动化验证)。

2.6 原始实现的不足

尽管朴素实现能够正确求解, 但存在以下局限, 构成本实验后续优化的入手点:

不足	说明
时间复杂度高	每个窗口扫描 k 个元素, 总计 $O(n \times k)$, n 、 k 较大时性能差
重复计算	相邻窗口共享 $k-1$ 个元素, 朴素法则重复比较, 未利用重叠结构
工程化不足	缺少统一输入校验、错误提示及可扩展的求解入口
代码规范待改进	首次 Pylint 检查会暴露 docstring、重复代码等问题 (见第三章)

2.7 与后续章节的关系

- **第三章 Pylint**: 对 `before_pylint/` 快照及本文件进行静态分析, 首次得分约 9.06/10, 修复后达 10.00/10;
- **第四章 Profile**: 对 `max_sliding_window_naive()` 进行性能分析, 墙钟 benchmark 测试与 profile 均显示热点集中在暴力双重循环;
- **第五章单元测试**: `TestNaiveModule` 与 `TestNaiveOptimizedConsistency` 验证本文件与优化实现结果一致;
- **主程序优化版**: `sliding_window_max.py` 中的 `max_sliding_window()` 以单调双端队列替代上述双重循环, 复杂度降为 $O(n)$ 。

3 Pylint 代码分析

静态代码分析 (Static Code Analysis) 是在不运行程序的前提下, 对源代码进行自动化检查的技术。本实验使用 **Pylint** 对 Python 源代码进行规范性与质量分析, 参考标准为 **PEP 8** (Python Enhancement Proposal 8)。Pylint 可从命名、格式、文档字符串、代码结构等多个维度给出评分与改进建议, 帮助我们发现并修正不符合规范的问题。

本章分析对象为主程序 `src/sliding_window_max.py` 与原始代码 `src/sliding_window_max_naive.py`。分析过程分为**首次检查 (修复前)**与**修复后检查**两阶段。

3.1 检查命令与配置

3.1.1 采用的规范

本实验以 **PEP 8** 作为编码规范依据。PEP 8 是 Python 官方社区广泛采纳的风格指南, 对缩进 (4 空格)、行宽、命名 (函数/变量 `snake_case`、类名 `CapWords`)、文档字符串、导入顺序等均有明确建议。Pylint 内置对 PEP 8 相关规则的检查, 并额外提供 Refactor、Warning、Error 等类别的诊断信息。

3.1.2 Pylint 配置文件

为适配本实验代码特点, 我们在 `src/.pylintrc` 中进行如下配置:

配置项	取值	说明
py-version	3.8	与本地 Python 版本一致
max-line-length	100	行宽上限 (算法代码略放宽)
good-names	i, j, k, n, idx	允许算法常用短变量名
max-returns	6	单函数 return 语句上限
disable	duplicate-code	全局禁用重复代码告警 (朴素版与主程序保留相似结构)

3.1.3 检查命令

基础检查（修复后代码，使用配置文件）：

在项目根目录执行：

```
1 pylint --rcfile=src/.pylintrc src/sliding_window_max.py
   src/sliding_window_max_naive.py
```

首次检查（修复前快照，不使用 `.pylintrc`，以复现原始问题列表）：

```
1 pylint src/before_pylint/sliding_window_max.py
   src/before_pylint/sliding_window_max_naive.py
```

详细统计报告（`-ry` 参数）：

```
1 python src/run_pylint.py
```

该脚本调用 `pylint.lint.Run(['-ry', ...])`，输出按类型统计、Raw metrics、Messages by category 等详细表格。

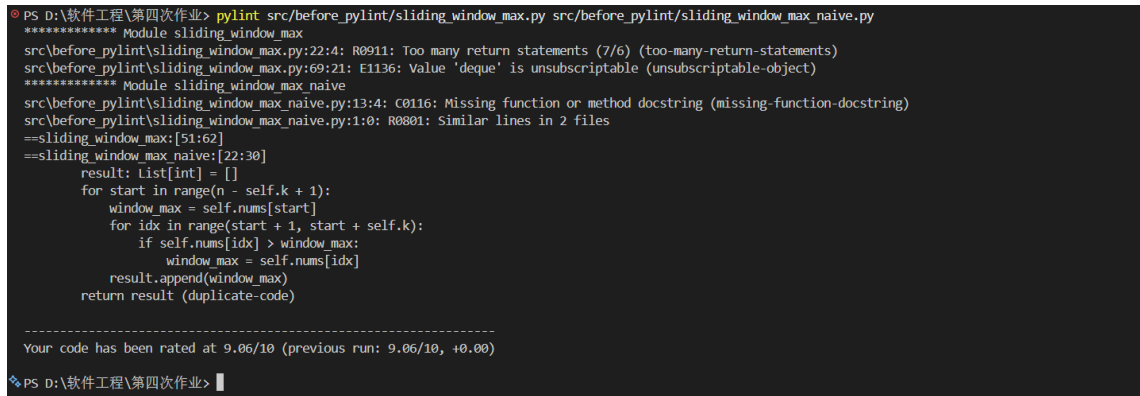
3.2 首次检查结果与问题修复

3.2.1 首次检查概况

对 `before_pylint/` 快照运行 Pylint，得到如下结果：

```
1 Your code has been rated at 9.06/10
```

共报告 4 条告警，涉及 Convention、Refactor、Error 三类。终端输出如下所示。



```
PS D:\软件工程\第四次作业> pylint src/before_pylint/sliding_window_max.py src/before_pylint/sliding_window_max_naive.py
***** Module sliding_window_max
src/before_pylint/sliding_window_max.py:22:4: R0911: Too many return statements (7/6) (too-many-return-statements)
src/before_pylint/sliding_window_max.py:69:21: E1136: Value 'deque' is unsubscriptable (unsubscriptable-object)
***** Module sliding_window_max_naive
src/before_pylint/sliding_window_max_naive.py:13:4: C0116: Missing function or method docstring (missing-function-docstring)
src/before_pylint/sliding_window_max_naive.py:1:0: R0801: Similar lines in 2 files
==sliding_window_max:[51:62]
==sliding_window_max_naive:[22:30]
    result: List[int] = []
    for start in range(n - self.k + 1):
        window_max = self.nums[start]
        for idx in range(start + 1, start + self.k):
            if self.nums[idx] > window_max:
                window_max = self.nums[idx]
        result.append(window_max)
    return result (duplicate-code)

-----
Your code has been rated at 9.06/10 (previous run: 9.06/10, +0.00)

PS D:\软件工程\第四次作业>
```

图 3.1: 图 1 首次 Pylint 检查结果 (9.06/10)

3.2.2 问题清单与修复措施

消息 ID	类别	问题描述	所在位置	修复方式
R0911	Refactor	函数中 return 语句过多 (7/6, too-many-return-statements)	sliding_window_max.py 第 22 行 array_is_legal()	将多个独立 if-return 改为 if-elif-else 链: 先确定错误信息变量 message, 再统一 print 并 return False, 合法路径单独 return True, 将 return 减至 2 处
E1136	Error	Value 'deque' is unsubscriptable (unsubscriptable-object)	sliding_window_max.py 第 69 行	Python 3.8 不支持 deque[int] 内置泛型下标, 改为 from typing import Deque, 注解写为 Deque[int]
C0116	Convention	缺少函数 docstring (missing-function-docstring)	sliding_window_max_naive.py 第 13 行 set_array()	补充「设置待处理数组与窗口大小。」
R0801	Refactor	两文件间存在相似代码 (duplicate-code)	sliding_window_max 与 sliding_window_max_naive 中朴素双重循环	在 naive 文件顶部添加 # pylint: disable=duplicate-code; 同时在 .pylintrc 中全局 disable=duplicate-code。此为刻意保留的对比基线, 非冗余复制

3.2.3 典型问题说明

R0911 — 过多 return 语句

修复前, `array_is_legal()` 对每个非法条件分别 `print` 并 `return False`, 共 7 处 `return`, 超过 `.pylintrc` 中 `max-returns=6` 的限制。重构后逻辑更清晰: 先判定错误类型并赋值提示信息, 仅在不满足任何错误条件时返回 `True`。

E1136 — deque 不可下标

Python 3.9+ 才支持 `collections.deque[int]` 形式。本实验环境为 Python 3.8, 须使用 `typing.Deque[int]`, 否则 Pylint 报类型注解错误。

R0801 — 重复代码

主程序中 `max_sliding_window_naive()` 与第二章原始文件的核心循环一致。为支持 Profile 对比与「原始代码」章节展示, 不宜合并删除; 通过 `pylint` 指令与配置文件合理抑制该告警, 并在报告中说明原因。

3.3 修复后结果

3.3.1 基础检查: 满分

对修复后的 `src/sliding_window_max.py` 与 `src/sliding_window_max_naive.py` 执行:

```
1 pylint --rcfile=src/.pylintrc src/sliding_window_max.py
   src/sliding_window_max_naive.py
```

得到:

```
1 Your code has been rated at 10.00/10 (previous run: 9.06/10, +0.94)
```

Convention、Refactor、Warning、Error 均为 0。终端输出如下所示。

```

PS D:\软件工程\第四次作业> pylint --rcfile=src/.pylintrc src/sliding_window_max.py src/sliding_window_max_naive.py

Your code has been rated at 10.00/10 (previous run: 9.06/10, +0.94)

PS D:\软件工程\第四次作业>
  
```

图 3.2: 图 2 修复后 Pylint 检查结果 (10.00/10)

3.3.2 详细报告 (-ry) 统计摘要

运行 `python src/run_pylint.py` 后，主要指标如下。

Statistics by type:

type	number	%documented	%badname
module	2	100.00	0.00
class	2	100.00	0.00
method	9	100.00	0.00
function	3	100.00	0.00

```

PS D:\软件工程\第四次作业> python src/run_pylint.py

Report
=====
103 statements analysed.

Statistics by type
-----
+-----+-----+-----+-----+-----+-----+
|type|number|old number|difference| %documented | %badname |
+-----+-----+-----+-----+-----+-----+
|module|2|2|=|100.00|0.00|
+-----+-----+-----+-----+-----+-----+
|class|2|2|=|100.00|0.00|
+-----+-----+-----+-----+-----+-----+
|method|9|9|=|100.00|0.00|
+-----+-----+-----+-----+-----+-----+
|function|3|3|=|100.00|0.00|
+-----+-----+-----+-----+-----+-----+

158 lines have been analyzed
  
```

图 3.3: 图 3 Pylint 详细报告: Statistics by type

Raw metrics: 共分析 158 行、103 条语句; 其中 code 68.35%、docstring 9.49%、empty 21.52%。

Duplication: 重复行数 0, 重复率 0.000% (配置禁用 duplicate-code 后统计为 0)。

```

158 lines have been analyzed

Raw metrics
-----
+-----+-----+-----+-----+-----+
|type    |number| %    |previous|difference|
+-----+-----+-----+-----+-----+
|code    |108   |68.35|NC      |NC        |
+-----+-----+-----+-----+-----+
|docstring|15    |9.49 |NC      |NC        |
+-----+-----+-----+-----+-----+
|comment  |1     |0.63 |NC      |NC        |
+-----+-----+-----+-----+-----+
|empty    |34    |21.52|NC      |NC        |
+-----+-----+-----+-----+-----+

Duplication
-----
+-----+-----+-----+-----+
|              |now|previous|difference|
+-----+-----+-----+-----+
|nb duplicated lines|0|0|0|
+-----+-----+-----+-----+
|percent duplicated lines|0.000|0.000|=|
+-----+-----+-----+-----+

```

图 3.4: 图 4 Pylint 详细报告: Raw metrics 与 Duplication

Messages by category:

type	number
convention	0
refactor	0
warning	0
error	0

```

Messages by category
-----
+-----+-----+-----+-----+
|type    |number|previous|difference|
+-----+-----+-----+-----+
|convention|0     |0       |0         |
+-----+-----+-----+-----+
|refactor  |0     |0       |0         |
+-----+-----+-----+-----+
|warning   |0     |0       |0         |
+-----+-----+-----+-----+
|error     |0     |0       |0         |
+-----+-----+-----+-----+

Messages
-----
+-----+-----+
|message id|occurrences|
+=====+=====+

-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

```

图 3.5: 图 5 Pylint 详细报告: Messages by category 与最终评分 10.00/10

3.4 主要代码修改对照

下面我们列出与 Pylint 相关的关键修改:

- (1) `array_is_legal()` 重构 (解决 R0911)

```
1 # 修复思路: if-elif-else 确定 message, 统一返回
2 if not isinstance(self.nums, list):
3     message = "错误: 输入必须是列表类型。"
4 elif len(self.nums) == 0:
5     message = "错误: 数组不能为空。"
6 # ... 其余 elif 分支 ...
7 else:
8     return True
9 print(message)
10 return False
```

(2) 类型注解 (解决 E1136)

```
1 from typing import Deque, List, Optional
2 # ...
3 index_queue: Deque[int] = deque()
```

(3) 补全 docstring (解决 C0116)

为 `SlidingWindowMaxNaive.set_array()` 增加函数说明字符串。

4 Profile 性能分析

在完成编码规范检查之后,需要验证算法优化是否带来可测量的性能提升。性能分析 (Performance Profiling) 旨在定位程序运行时的耗时热点、统计函数调用次数,并为复杂度理论提供实验支撑。本实验采用 Python 标准库中的 `profile` 模块进行剖面分析,并结合 `time.perf_counter()` 进行墙钟时间 (Wall-clock Time) 对比。

分析对象为第一章所述的两种实现: 朴素暴力法 (`max_sliding_window_naive()`) 与单调双端队列优化法 (`max_sliding_window()`)。

4.1 分析方法

4.1.1 性能分析工具

工具	用途	本实验中的使用方式
<code>profile</code> 模块	记录各函数调用次数与耗时	<code>profile.run("swm.profile_test(scale, use_naive=...)")</code>
<code>time.perf_counter()</code>	高精度墙钟计时	<code>src/benchmark_time.py</code> 中对同一输入分别计时
<code>profile_test()</code>	统一性能测试入口	定义于 <code>src/sliding_window_max.py</code>

`profile` 与 `cProfile` 功能类似,输出格式更易读。输出报告中的关键字段含义如下:

字段	含义
ncalls	函数被调用次数
tottime	函数自身耗时（不含子函数）
cumtime	累计耗时（含子函数）
percall	每次调用平均耗时

4.1.2 测试数据生成

`profile_test(scale, use_naive)` 的构造方式：

- 生成长度为 `scale` 的整数数组：`nums[i] = i % 100 - 50`（取值范围 $[-50, 49]$ ）；
- 窗口大小：`k = min(1000, max(1, scale // 10))`；
- 为聚焦核心算法，测试时直接设置 `solver.legal = True`，跳过 `array_is_legal()` 的全量校验（输入合法性由第五章单元测试保证）。

4.1.3 实验命令

墙钟时间对比（同规模公平对比）：

```
1 python src/benchmark_time.py
```

朴素算法 Profile（规模 30000）：

```
1 python src/run_profile_naive_only.py
```

优化算法 Profile（规模 500000）：

```
1 python src/run_profile_optimized_only.py
```

一键运行（墙钟表 + 两次 Profile）：

```
1 python src/run_profile.py
```

4.1.4 复杂度理论基准

实现	时间复杂度	空间复杂度
朴素暴力法	$O(n \times k)$	$O(1)$
单调双端队列	$O(n)$	$O(k)$

后续墙钟对比在**相同** n 、 k 下进行；Profile 则对两种算法采用**不同规模**（朴素用较小规模以避免过长等待，优化用较大规模以观察渐近行为），解读时需注意，不能将两种 Profile 的绝对耗时直接横向比较。

4.2 墙钟时间对比

4.2.1 测试环境与方法

- **环境：** Windows, Python 3.8;
- **方法：**对每一组 `scale`, 使用相同 `nums` 与 `k`, 分别调用朴素版与优化版各一次, 用 `time.perf_counter()` 测量墙钟时间;
- **公平性：**两种算法处理完全相同的输入, 可直接比较耗时与加速比。

4.2.2 实测结果

scale (n)	k	朴素 (s)	优化 (s)	加速比
5000	500	0.135259	0.003007	45.0 ×
10000	1000	0.500676	0.005437	92.1 ×
20000	1000	1.101932	0.010727	102.7 ×
50000	1000	2.730823	0.023522	116.1 ×

终端输出如下所示。

```
PS D:\软件工程\第四次作业> python src/benchmark_time.py
scale | k | naive(s) | optimized(s) | speedup
-----|---|-----|-----|-----
5000 | 500 | 0.135259 | 0.003007 | 45.0x
10000 | 1000 | 0.500676 | 0.005437 | 92.1x
20000 | 1000 | 1.101932 | 0.010727 | 102.7x
50000 | 1000 | 2.730823 | 0.023522 | 116.1x
PS D:\软件工程\第四次作业>
```

图 4.6: 图 6 墙钟时间对比 (benchmark_time.py)

4.2.3 结果分析

1. **朴素算法随规模近似线性恶化：**当 n 从 5000 增至 50000 (k 相应从 500 增至 1000), 耗时从约 0.14s 增至约 2.73s, 增长约 20 倍, 与 $O(n \times k)$ 理论一致。
2. **优化算法增长极为缓慢：**同等规模下, 优化版耗时仅从约 0.003s 增至约 0.024s, 体现 $O(n)$ 的优势。
3. **加速比随规模增大而提升：**从 45 $\times$ 升至 116 $\times$, 说明数据规模越大, 单调队列优化相对暴力法的收益越明显。
4. **工程意义：**在 $n = 50000$ 、 $k = 1000$ 时, 朴素法需约 **2.73s**, 优化法仅约 **0.024s**, 若应用于实时数据流场景, 优化版更具实用价值。

4.3 Profile 结果解读

4.3.1 朴素算法 Profile (scale = 30000, $k = 3000$)

运行 `python src/run_profile_naive_only.py`, 得到:

```

1 29013 function calls in 1.844 seconds
2 sliding_window_max.py:41(max_sliding_window_naive)
3     tottime = 1.828s      cumtime = 1.844s

```

```

PS D:\软件工程\第四次作业> python src/run_profile_naive_only.py
朴素算法 profile_test(scale=30000)
29013 function calls in 1.844 seconds

Ordered by: standard name

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
29001   0.016   0.000   0.016   0.000 :0(append)
1       0.000   0.000   1.844   1.844 :0(exec)
1       0.000   0.000   0.000   0.000 :0(len)
1       0.000   0.000   0.000   0.000 :0(max)
1       0.000   0.000   0.000   0.000 :0(min)
1       0.000   0.000   0.000   0.000 :0(setprofile)
1       0.000   0.000   1.844   1.844 <string>:1(<module>)
0       0.000   0.000   0.000   0.000 profile:0(profiler)
1       0.000   0.000   1.844   1.844 profile:0(swm.profile_test(30000, use_naive=True))
1       0.000   0.000   0.000   0.000 sliding_window_max.py:10(__init__)
1       1.828   1.828   1.844   1.844 sliding_window_max.py:41(max_sliding_window_naive)
1       0.000   0.000   1.844   1.844 sliding_window_max.py:80(solve)
1       0.000   0.000   1.844   1.844 sliding_window_max.py:94(profile_test)
1       0.000   0.000   0.000   0.000 sliding_window_max.py:96(<listcomp>)

```

图 4.7: 图 7 朴素算法 Profile 分析 (scale=30000)

解读:

- 总耗时 1.844s 中, `max_sliding_window_naive()` 的 `tottime` 达 1.828s, 占总量 99% 以上;
- 热点完全落在双重循环暴力枚举上, 与第二章原始代码分析一致;
- `append` 调用约 29001 次, 对应结果列表写入, 开销可忽略;
- **结论:** 朴素实现的性能瓶颈明确, 优化应针对该函数的算法结构, 而非外围逻辑。

4.3.2 优化算法 Profile (scale = 500000, $k = 1000$)

运行 `python src/run_profile_optimized_only.py`, 得到:

```

1 1499012 function calls in 3.922 seconds
2 sliding_window_max.py:57(max_sliding_window)    tottime=2.047s  cumtime=3.844s
3 :0(append)      ncalls=999001    tottime=1.172s
4 :0(pop)         ncalls=499999    tottime=0.625s

```

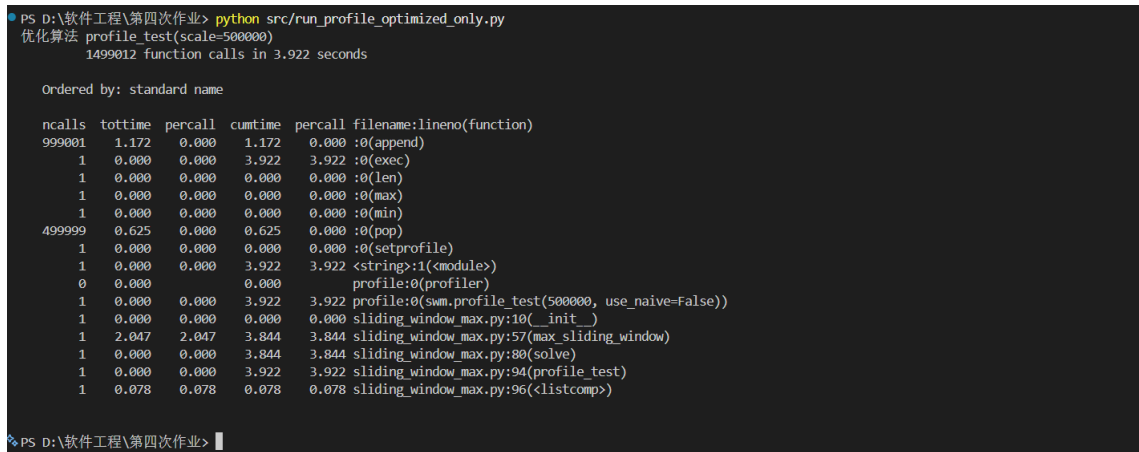


图 4.8: 图 8 优化算法 Profile 分析 (scale=500000)

解读:

- 总调用次数约 **150 万次**，总耗时 **3.922s**（规模为 50 万，大于墙钟测试，用于观察热点分布）；
- 核心函数 `max_sliding_window()` 的 `tottime` 为 **2.047s**，`cumtime` 为 **3.844s**，仍是主要耗时点；
- 双端队列操作：`append` 约 99 万次 (1.172s)，`pop` 约 50 万次 (0.625s)，说明 Python 层 deque 操作占相当比重；
- 与墙钟对比：虽本次 Profile 总耗时约 3.9s，但需注意规模为 500000；墙钟表显示在 相同规模 $n = 50000$ 时，朴素法约 2.73s 而优化法仅约 0.024s，同规模下优化版仍远快于朴素版；
- 结论：优化后热点仍在核心求解函数，但单位数据耗时显著降低。

4.4 优化总结

4.4.1 优化前后对照

阶段	实现	时间复杂度	实测 ($n = 50000$, $k = 1000$)
优化前	<code>sliding_window_max_naive.py</code> / <code>max_sliding_window_naive()</code>	$O(n \times k)$	墙钟约 2.73s
优化后	<code>max_sliding_window()</code> 单调双端队列	$O(n)$	墙钟约 0.024s
提升	—	理论降阶	加速约 116×

4.4.2 优化思路回顾

1. 识别瓶颈：Profile 显示朴素版 99% 时间花在双重循环；
2. 算法改进：引入单调双端队列，维护可能成为最大值的递减下标序列，每个元素最多入队、出队各一次；
3. 验证效果：墙钟对比与 Profile 均表明优化有效；第五章单元测试保证两种实现结果一致。

4.5 本章小结与作业问题 e) 对应

报告问题 e)	本实验回答
算法复杂度是多少?	朴素 $O(n \times k)$, 优化 $O(n)$, 空间分别为 $O(1)$ 与 $O(k)$
如何对代码性能进行分析?	profile 剖面分析 定位热点 + time.perf_counter() 墙钟对比 量化加速比
分析结果如何?	朴素热点在 <code>max_sliding_window_naive</code> ; 优化后单位数据耗时大幅下降, 加速比 $45\times \sim 116\times$
如何进行优化?	暴力枚举 $\rightarrow$ 单调双端队列; 每个元素至多进出队一次, 避免重复比较

5 Unittest 单元测试

单元测试 (Unit Testing) 是软件工程中验证各模块功能正确性的重要手段。本实验使用 Python 标准库 `unittest` 框架, 对滑动窗口最大值求解程序进行系统化的**黑盒测试**与**白盒测试**, 覆盖功能正确性、边界条件、异常输入及朴素/优化算法一致性等方面。

测试代码位于 `src/test_sliding_window_max.py`, 测试数据来自 `data/test_cases.json`。本章对应报告问题 f) 中的「测试用例设计思路、测试用例表、测试通过率」。

5.1 测试设计思路

5.1.1 测试策略：黑盒与白盒相结合

本实验采用**黑盒测试**（不关注内部实现，仅验证输入—输出）与**白盒测试**（针对内部分支与异常路径）相结合的策略：

测试类型	目的	本实验中的体现
黑盒测试	验证功能是否符合规格	给定 <code>nums</code> 、 <code>k</code> , 断言输出与期望值一致
白盒测试	覆盖内部分支与异常处理	非法输入时 <code>legal=False</code> 、 <code>solve()</code> 返回 <code>None</code>
回归/一致性测试	优化前后结果应相同	朴素与优化算法对同一输入输出一致

5.1.2 黑盒测试设计维度

维度	设计考虑	测试类 / 方法
数值正负	全正、全负、正负混合、含 0	test_all_positive、test_all_negative、test_mixed_sign_with_zero
边界条件	$k = 1$ 、 $k = n$ 、单元素数组	JSON 用例 + test_single_element
典型功能	题目示例、单调增/减、重复最大值	test_official_example、test_cases_from_json、test_duplicate_max_values
算法一致性	朴素与优化结果完全相同	TestNaiveOptimizedConsistency
原始代码	独立文件 sliding_window_max_naive.py	TestNaiveModule

5.1.3 白盒测试设计维度

场景	期望行为	测试方法
空数组	legal=False, 提示「不能为空」	test_empty_array
$k = 0$ 、 $k < 0$	legal=False, 提示「正整数」	test_k_zero、test_k_negative
$k > n$	legal=False, 提示「不能大于」	test_k_greater_than_n
非 list 类型	array_is_legal() 返回 False	test_non_list_input
非 int 元素 / 非 int 的 k	legal=False	test_non_int_element、test_non_int_k
非法输入调用 solve()	返回 None	test_solve_returns_none_when_illegal
非法输入调用朴素方法	返回 None 并打印提示	test_naive_method_returns_none_when_illegal

白盒测试中，使用 `unittest.mock.patch` 重定向 `sys.stdout`，避免错误提示信息干扰终端输出，同时可用 `assertIn` 验证提示文案。

5.1.4 测试类结构

测试类	职责	测试方法数
TestSlidingWindowFunctional	优化算法功能正确性	7
TestNaiveOptimizedConsistency	朴素与优化结果一致	2
TestSlidingWindowValidation	输入校验与异常分支	9
TestNaiveModule	原始独立模块	2
合计	—	20

5.2 测试用例表

下表汇总本实验设计的代表性测试用例（完整数据见于 `data/test_cases.json`）。

序号	测试样例描述	输入 nums	k	期望输出 / 期望行为	测试类型
1	题目示例	[1,3,-1,-3,5,3,6,7]	3	[3,3,5,5,6,7]	黑盒-功能
2	窗口大小为 1	[4,2,9,1]	1	[4,2,9,1]	黑盒-边界
3	窗口等于数组长度	[2,1,5,3]	4	[5]	黑盒-边界
4	全为负数	[-1,-3,-2,-5]	2	[-1,-2,-2]	黑盒-功能
5	单调递增	[1,2,3,4,5]	3	[3,4,5]	黑盒-功能
6	单调递减	[5,4,3,2,1]	3	[5,4,3]	黑盒-功能
7	全正整数	[3,4,5,6]	2	[4,5,6]	黑盒-功能
8	含 0 的混合序列	[-2,0,-1,3]	2	[0,0,3]	黑盒-功能
9	单元素数组	[7]	1	[7]	黑盒-边界
10	重复最大值	[1,3,3,2]	3	[3,3]	黑盒-功能
11	朴素与优化结果一致	JSON 全部 6 组用例	—	两者相等	黑盒-一致性
12	空数组	[]	1	非法, solve 返回 None	白盒-异常
13	$k = 0$	[1,2]	0	非法	白盒-异常
14	$k > n$	[1,2,3]	4	非法	白盒-异常
15	非 list 输入	tuple	2	非法	白盒-异常
16	非 int 元素	[1,2.5,3]	2	非法	白盒-异常
17	朴素独立模块官方例	[1,3,-1,-3,5,3,6,7]	3	[3,3,5,5,6,7]	黑盒-原始代码

说明：序号 1~6 由 `test_cases_from_json` 通过 `subTest` 批量执行；序号 11 对 JSON 中每组用例分别比较朴素与优化输出。

5.3 测试代码说明

5.3.1 测试文件与运行方式

测试文件：`src/test_sliding_window_max.py`

```

1 # 运行全部单元测试 (verbose 输出)
2 python src/test_sliding_window_max.py
3
4 # 或使用 unittest 模块
5 python -m unittest src.test_sliding_window_max -v

```

辅助入口函数 `unit_test(nums, k, use_naive=False)` 定义于 `sliding_window_max.py`, 封装「创建求解器 → 设置输入 → 调用 solve」流程, 便于测试代码简洁调用。

5.3.2 代表性测试代码

(a) JSON 数据驱动黑盒测试

```

1 def test_cases_from_json(self):
2     for case in load_json_cases():
3         with self.subTest(description=case["description"]):
4             result = swm.unit_test(case["nums"], case["k"], use_naive=False)
5             self.assertEqual(result, case["expected"])

```

(b) 朴素与优化一致性测试

```

1 def test_consistency_on_json_cases(self):
2     for case in load_json_cases():
3         with self.subTest(description=case["description"]):
4             naive = swm.unit_test(case["nums"], case["k"], use_naive=True)
5             optimized = swm.unit_test(case["nums"], case["k"], use_naive=False)
6             self.assertEqual(naive, optimized)
7             self.assertEqual(optimized, case["expected"])

```

(c) 白盒异常测试 (含 stdout 捕获)

```

1 def test_empty_array(self):
2     with patch("sys.stdout", new_callable=StringIO) as mock_out:
3         self.solver.set_array([], 1)
4         self.assertFalse(self.solver.legal)
5         self.assertIn("不能为空", mock_out.getvalue())

```

5.3.3 测试与工程实践的对应

- 使用 `subTest`: JSON 中某一组失败时, 不影响其余用例执行, 便于定位问题;
- 使用 `setUp`: `TestSlidingWindowValidation` 在每个测试方法前创建新的 `SlidingWindowMax` 实例, 保证用例隔离;
- 测试数据与代码分离: `data/test_cases.json` 便于增删用例而无需修改测试逻辑。

5.4 测试运行结果

5.4.1 执行结果

在项目根目录运行 `python src/test_sliding_window_max.py`，得到：

```

1 -----
2 Ran 20 tests in 0.015s
3
4 OK

```

全部 20 个测试方法均通过，测试通过率 100%。终端输出如下所示。

```

PS D:\软件工程\第四次作业> python src/test_sliding_window_max.py
全负整数。 ... ok
test_all_positive (__main__.TestSlidingWindowFunctional)
全正整数。 ... ok
test_cases_from_json (__main__.TestSlidingWindowFunctional)
遍历 JSON 中全部标准用例。 ... ok
test_duplicate_max_values (__main__.TestSlidingWindowFunctional)
窗口内存在重复最大值 (n=4, k=3 共 2 个窗口)。 ... ok
test_mixed_sign_with_zero (__main__.TestSlidingWindowFunctional)
正负混合且含 0。 ... ok
test_official_example (__main__.TestSlidingWindowFunctional)
官方示例。 ... ok
test_single_element (__main__.TestSlidingWindowFunctional)
数组长度为 1。 ... ok
test_empty_array (__main__.TestSlidingWindowValidation)
空数组应判定非法。 ... ok
test_k_greater_than_n (__main__.TestSlidingWindowValidation)
k > len(nums) 应判定非法。 ... ok
test_k_negative (__main__.TestSlidingWindowValidation)
k < 0 应判定非法。 ... ok
test_k_zero (__main__.TestSlidingWindowValidation)
k = 0 应判定非法。 ... ok
test_naive_method_returns_none_when_illegal (__main__.TestSlidingWindowValidation)
非法输入时朴素方法也返回 None (覆盖错误分支)。 ... ok
test_non_int_element (__main__.TestSlidingWindowValidation)
非 int 元素应判定非法。 ... ok
test_non_int_k (__main__.TestSlidingWindowValidation)
k 非 int 应判定非法。 ... ok
test_non_list_input (__main__.TestSlidingWindowValidation)
非 list 类型应判定非法。 ... ok
test_solve_returns_none_when_illegal (__main__.TestSlidingWindowValidation)
非法输入时 solve 返回 None。 ... ok
-----
Ran 20 tests in 0.015s
OK
PS D:\软件工程\第四次作业>

```

图 5.9: 图 9 单元测试全部通过 (20 tests OK)

5.4.2 结果汇总

指标	结果
测试类数	4
测试方法数	20
通过	20
失败	0
错误	0
测试通过率	100%
墙钟耗时	约 0.015s

此外，`test_cases_from_json` 与 `test_consistency_on_json_cases` 内部各含 6 个 `subTest`，实际执行的断言项多于 20，但 unittest 统计以 test 方法为单位。

5.5 本章小结与作业问题 f) 对应（前半）

报告问题 f)	本实验回答
测试用例设计思路	§5.1: 黑盒（正负/边界/功能/一致性）+ 白盒（非法输入分支）
测试用例表	§5.2 表（17 项代表性用例）
测试通过率	100%（20/20）

单元测试从功能与异常两个层面验证了程序的正确性与健壮性，并确认朴素算法与优化算法输出一致，为第四章的性能优化提供了**正确性保障**。下面第六章将进一步给出**语句覆盖率**统计及 HTML 覆盖率报告分析。

6 代码覆盖率

代码覆盖率（Code Coverage）衡量测试执行过程中**源代码被运行的程度**，是评价测试充分性的重要指标。本实验在第五章单元测试全部通过的基础上，使用 `coverage` 工具统计被测模块的**语句覆盖率（Statement Coverage）**，并生成终端摘要与 HTML 可视化报告。

6.1 覆盖率工具与命令

6.1.1 选用的覆盖指标

本实验选用 **语句覆盖率（Statement Coverage）** 作为覆盖指标。

项目	说明
定义	被执行到的语句条数占可执行语句总数的比例
公式	$\text{Coverage} = \frac{\text{已执行语句数}}{\text{可执行语句总数}} \times 100\%$
选择理由	<code>coverage</code> 工具默认统计语句覆盖；能直观反映测试对业务代码的触达程度

6.1.2 工具与配置文件

文件	作用
<code>coverage</code> (pip 包)	运行测试并记录语句执行轨迹
<code>src/.coveragerc</code>	指定统计范围、omit 测试脚本、HTML 输出目录
<code>src/run_coverage.py</code>	一键： <code>coverage run</code> → <code>report</code> → <code>html</code>

`.coveragerc` 中 `include` 仅统计 `sliding_window_max.py` 与 `sliding_window_max_naive.py`；`omit` 排除 `test_*.py`、`run_*.py` 等辅助脚本。

6.1.3 运行命令

一键生成：

```
1 python src/run_coverage.py
```

分步执行：

```
1 cd src
2 coverage run test_sliding_window_max.py
3 coverage report -m
4 coverage html -d ../my_coverage_result
```

HTML 报告目录：`my_coverage_result/index.html`（浏览器打开可查看逐行着色）。

6.2 覆盖率结果

6.2.1 执行过程

运行 `python src/run_coverage.py` 时，首先重新执行第五章全部 20 个单元测试（结果仍为 OK），再输出覆盖率摘要并生成 HTML。终端输出如下所示。

```
PS D:\软件工程\第四次作业> python src/run_coverage.py
1. 运行 coverage + unittest
=====
test_naive_module_invalid_k (__main__.TestNaiveModule)
朴素独立模块，非法 k 返回 None。 ... ok
test_naive_module_official_example (__main__.TestNaiveModule)
朴素独立模块，官方示例。 ... ok
test_consistency_on_json_cases (__main__.TestNaiveOptimizedConsistency)
JSON 用例上两种实现结果相同。 ... ok
test_consistency_random_pattern (__main__.TestNaiveOptimizedConsistency)
构造序列上两种实现一致。 ... ok
test_all_negative (__main__.TestSlidingWindowFunctional)
全负整数。 ... ok
test_all_positive (__main__.TestSlidingWindowFunctional)
全正整数。 ... ok
test_cases_from_json (__main__.TestSlidingWindowFunctional)
遍历 JSON 中全部标准用例。 ... ok
test_duplicate_max_values (__main__.TestSlidingWindowFunctional)
窗口内存在重复最大值 (n=4, k=3 共 2 个窗口)。 ... ok
test_mixed_sign_with_zero (__main__.TestSlidingWindowFunctional)
正负混合且含 0。 ... ok
test_official_example (__main__.TestSlidingWindowFunctional)
官方示例。 ... ok
test_single_element (__main__.TestSlidingWindowFunctional)
数组长度为 1。 ... ok
test_empty_array (__main__.TestSlidingWindowValidation)
空数组应判定非法。 ... ok
test_k_greater_than_n (__main__.TestSlidingWindowValidation)
k > len(nums) 应判定非法。 ... ok
test_k_negative (__main__.TestSlidingWindowValidation)
k < 0 应判定非法。 ... ok
test_k_zero (__main__.TestSlidingWindowValidation)
k = 0 应判定非法。 ... ok
test_naive_method_returns_none_when_illegal (__main__.TestSlidingWindowValidation)
非法输入时朴素方法也返回 None (覆盖错误分支)。 ... ok
```

图 6.10: 图 10 run_coverage 终端输出（1/2：单元测试重跑）

```

PS D:\软件工程\第四次作业> python src/run_coverage.py
k > len(nums) 应判定非法。 ... ok
test k negative (__main__.TestSlidingWindowValidation)
k<0 应判定非法。 ... ok
test k zero (__main__.TestSlidingWindowValidation)
k=0 应判定非法。 ... ok
test naive method returns none when illegal (__main__.TestSlidingWindowValidation)
非法输入时朴素方法也返回 None (覆盖错误分支)。 ... ok
test non_int_element (__main__.TestSlidingWindowValidation)
非 int 元素应判定非法。 ... ok
test non_int_k (__main__.TestSlidingWindowValidation)
k 非 int 应判定非法。 ... ok
test non_list_input (__main__.TestSlidingWindowValidation)
非 list 类型应判定非法。 ... ok
test solve returns none when illegal (__main__.TestSlidingWindowValidation)
非法输入时 solve 返回 None。 ... ok

-----
Ran 20 tests in 0.007s

OK

=====
2. 终端覆盖率摘要 (语句覆盖率)
=====
Name                               Stmts  Miss  Cover   Missing
-----
sliding_window_max.py              85     18  78.82%  96-102, 107-119, 123
sliding_window_max_naive.py         20      0 100.00%
-----
TOTAL                             105     18  82.86%

=====
3. 生成 HTML 报告
=====
Wrote HTML report to D:\软件工程\第四次作业\src\..\my_coverage_result\index.html
HTML 报告目录: D:\软件工程\第四次作业\my_coverage_result
PS D:\软件工程\第四次作业>

```

图 6.11: 图 11 run_coverage 终端输出 (2/2: 覆盖率摘要与 HTML 生成)

6.2.2 终端覆盖率摘要

模块	语句数 (Stmts)	未覆盖 (Miss)	语句覆盖率	未覆盖行号
sliding_window_max.py	85	18	78.82%	96-102, 107-119, 123
sliding_window_max_naive.py	20	0	100.00%	—
合计 (TOTAL)	105	18	82.86%	—

分析:

1. 原始代码模块 `sliding_window_max_naive.py` 达到 **100%** 语句覆盖, 说明针对第二章原始实现的测试用例 (`TestNaiveModule` 等) 已触及其全部可执行语句。
2. 主程序 `sliding_window_max.py` 为 **78.82%**, 未覆盖的 18 行集中于性能测试与演示入口 (详见 §6.3), 核心业务逻辑 (`max_sliding_window`、`array_is_legal`、`solve` 等) 已被第五章测试覆盖。
3. 整体 **82.86%** 在本实验场景下较为合理: 功能与异常路径已测, 刻意未测 `profile/main` 等辅助代码。

6.2.3 HTML 覆盖率报告总览

浏览器打开 `my_coverage_result/index.html`, 可查看文件级与函数级覆盖率汇总。

Coverage report: 82.86%

Files

Functions

Classes

filter...

hide covered

coverage.py v7.6.1, created at 2026-06-04 11:24 +0800

File ▲

	statements	missing	excluded	coverage
sliding_window_max_naive.py	20	0	0	100.00%
sliding_window_max.py	85	18	0	78.82%
Total	105	18	0	82.86%

coverage.py v7.6.1, created at 2026-06-04 11:24 +0800

图 6.12: 图 12 HTML 覆盖率总览 (index.html)

Coverage report: 82.86%

Files

Functions

Classes

filter...

hide covered

coverage.py v7.6.1, created at 2026-06-04 11:24 +0800

File	function	statements	missing	excluded	coverage
sliding_window_max_naive.py	SlidingWindowMaxNaive__init__	2	0	0	100.00%
sliding_window_max_naive.py	SlidingWindowMaxNaive.set_array	2	0	0	100.00%
sliding_window_max_naive.py	SlidingWindowMaxNaive.max_sliding_window	11	0	0	100.00%
sliding_window_max_naive.py	(no function)	5	0	0	100.00%
sliding_window_max.py	SlidingWindowMax__init__	3	0	0	100.00%
sliding_window_max.py	SlidingWindowMax.set_array	3	0	0	100.00%
sliding_window_max.py	SlidingWindowMax.array_is_legal	15	0	0	100.00%
sliding_window_max.py	SlidingWindowMax.max_sliding_window_naive	12	0	0	100.00%
sliding_window_max.py	SlidingWindowMax.max_sliding_window	15	0	0	100.00%
sliding_window_max.py	SlidingWindowMax.solve	3	0	0	100.00%
sliding_window_max.py	unit_test	3	0	0	100.00%
sliding_window_max.py	profile_test	7	7	0	0.00%
sliding_window_max.py	main	10	10	0	0.00%
sliding_window_max.py	(no function)	14	1	0	92.86%
Total		105	18	0	82.86%

coverage.py v7.6.1, created at 2026-06-04 11:24 +0800

图 6.13: 图 13 HTML 函数级覆盖率 (function_index.html)

图 12 展示各文件覆盖率百分比及缺失语句数；图 13 列出各函数/方法的覆盖情况，便于定位具体未覆盖函数。

6.3 未覆盖代码说明

点击 `sliding_window_max.py` 进入逐行报告 (`sliding_window_max_py.html`)，绿色行为已覆盖，红色行为未覆盖。

```

Coverage for sliding_window_max.py: 78.82%
85 statements 67 run 18 missing 0 excluded
« prev ^ index » next coverage.py v7.6.1, created at 2026-06-04 11:24 +0800

1 """滑动窗口最大值：提供朴素实现与单调队列优化实现。"""
2
3 from collections import deque
4 from typing import Deque, List, Optional
5
6
7 class SlidingWindowMax:
8     """滑动窗口最大值求解器。"""
9
10    def __init__(self) -> None:
11        """初始化数组与窗口大小。"""
12        self.nums: List[int] = []
13        self.k: int = 0
14        self.legal: bool = False
15
16    def set_array(self, nums: List[int], k: int) -> None:
17        """直接设置待处理数组与窗口大小。"""
18        self.nums = nums
19        self.k = k
20        self.legal = self.array_is_legal()
21
22    def array_is_legal(self) -> bool:
23        """判断输入数组与窗口大小是否合法。"""
24        if not isinstance(self.nums, list):
25            message = "错误：输入必须是列表类型。"
26            message = "错误：数组不能为空。"
27            elif len(self.nums) == 0:
28            elif not isinstance(self.k, int):
29                message = "错误：窗口大小 k 必须是整数。"

```

图 6.14: 图 14 已覆盖语句（绿色，共 67 行）

```

sliding_window_max.py: 78.82% 67 18 0
94 def profile_test(scale: int, use_naive: bool = False) -> None:
95     """性能分析调用的辅助函数（跳过校验以聚焦核心算法）。"""
96     nums = [i % 100 - 50 for i in range(scale)]
97     k = min(1000, max(1, scale // 10))
98     solver = SlidingWindowMax()
99     solver.nums = nums
100    solver.k = k
101    solver.legal = True
102    solver.solve(use_naive=use_naive)
103
104
105 def main() -> None:
106     """主函数：演示官方示例。"""
107     example_nums = [1, 3, -1, -3, 5, 3, 6, 7]
108     example_k = 3
109
110    solver = SlidingWindowMax()
111    solver.set_array(example_nums, example_k)
112
113    naive_result = solver.solve(use_naive=True)
114    optimized_result = solver.solve(use_naive=False)
115
116    print("输入 nums:", example_nums)
117    print("窗口大小 k:", example_k)
118    print("朴素算法结果:", naive_result)
119    print("优化算法结果:", optimized_result)
120
121
122 if __name__ == "__main__":
123     main()

```

图 6.15: 图 15 未覆盖语句（红色，共 18 行）

未覆盖行号与原因：

行号区间	所属代码	未覆盖原因
96 ~ 102	profile_test()	第四章性能分析专用入口，单元测试不调用
107 ~ 119	main()	命令行演示题目示例，测试以模块导入方式运行
123	if __name__ == "__main__":	测试执行路径不经过模块主入口

已覆盖的核心逻辑（图 14 绿色部分）包括：

- SlidingWindowMax 构造与 set_array()；
- array_is_legal() 全部校验分支（第五章白盒测试）；
- max_sliding_window_naive()、max_sliding_window()、solve()；

- `unit_test()` 辅助函数。

`profile_test`、`main` 等辅助入口可不纳入功能单测覆盖范围；未覆盖行不影响对算法正确性的评价。

覆盖率与测试通过率的关系

指标	数值	含义
测试通过率	100% (20/20)	所有断言均通过，无功能错误
语句覆盖率	82.86%	大部分业务语句被测试执行到
关系	高通过率 + 较高覆盖率	测试既 正确 又 较充分 ；未覆盖部分为已知、可解释的辅助代码

6.4 缺陷报告

6.4.1 被测程序缺陷

缺陷编号	描述	严重程度	状态
—	单元测试与覆盖率运行过程中 未发现 被测程序功能缺陷	—	无 open 缺陷

6.4.2 测试与用例相关缺陷

缺陷编号	类型	描述	处理
TC-002	覆盖率预期	主程序未覆盖 <code>profile_test/main</code> 属 预期行为 ，非程序缺陷	在 §6.3 说明，不列为缺陷

本实验 82.86% 合理满足要求，且未覆盖部分均有明确工程理由。

6.5 本章小结与作业问题 f) 对应

报告问题 f)	本实验回答
覆盖率（覆盖指标）	语句覆盖率 ，合计 82.86%（主模块 78.82%，naive 100%）
测试通过率	100% (20/20)
缺陷报告	§6.4：无程序缺陷

第五、六章共同完成了课程对单元测试与覆盖率的要求：测试**全部通过**，覆盖率达到合理水平，未覆盖代码可解释，缺陷记录完整。

7 编程规范

本章从规范遵循、参考标准与检查方法三方面，简要回答作业要求 a)。第三章已给出 Pylint 的详细过程与截图，此处侧重**结论**及与本项目实现的结合，不再重复展开。

7.1 是否遵守编程规范？

结论：是，本实验代码遵守编程规范。

依据如下：

1. 使用 **Pylint** 对 `src/sliding_window_max.py`、`src/sliding_window_max_naive.py` 进行静态分析，修复问题后评分为 **10.00/10**，Convention / Refactor / Warning / Error 均为 0。
2. 全部类、方法及模块均有 **docstring**，文档覆盖率 100%。
3. 命名、缩进、行宽等符合 **PEP 8** 要求；算法中常用的 `i`、`j`、`k`、`n`、`idx` 等在 `.pylintrc` 中声明为合法短名。

本实验在开发过程中遵循「编写 → Pylint 检查 → 按提示修改 → 再检查」的流程，而非事后一次性检查。

7.2 参考的规范

本实验参考 **PEP 8** (*Style Guide for Python Code*)，即 Python 官方推荐的编码风格指南。与本项目直接相关的要点包括：

PEP 8 要点	本实验中的体现
缩进	统一 4 空格，不使用 Tab
行宽	<code>.pylintrc</code> 设 <code>max-line-length=100</code> （算法代码略宽于默认 79，仍保持可读）
命名	函数/变量 <code>snake_case</code> （如 <code>max_sliding_window</code> 、 <code>set_array</code> ）；类名 <code>CapWords</code> （如 <code>SlidingWindowMax</code> ）
文档字符串	模块、类、公开方法均写 <code>docstring</code> ，说明功能与复杂度
导入	标准库在前（ <code>collections</code> 、 <code>typing</code> ），同目录模块通过测试文件导入
空行	类定义、方法之间适当空行，结构清晰

7.3 如何检查是否遵守编程规范？

本实验采用 **Pylint** + **配置文件** 进行自动化检查，主要步骤如下：

(1) 命令行检查

```
1 pylint --rcfile=src/.pylintrc src/sliding_window_max.py
   src/sliding_window_max_naive.py
```

(2) 详细报告（可选）

```
1 python src/run_pylint.py
```

使用 `-ry` 参数输出类型统计、Raw metrics、Messages by category 等。

(3) 配置说明

`src/.pylintrc` 指定 Python 3.8、行宽、合法短变量名、单函数 `return` 上限等，使检查结果既严格又适配本实验代码特点。

7.4 小结：作业问题 a) 一览

问题	回答
是否遵守编程规范？	是；Pylint 10.00/10，无规范类告警
参考哪个规范？	PEP 8
如何检查？	<code>pylint -rcfile=src/.pylintrc</code> ；辅以 <code>python src/run_pylint.py</code> 生成详细报告

规范检查与第三章实践相呼应：第三章展示**过程与证据**，本章给出**针对 a) 的简明结论**。

8 程序可扩展性

程序的可扩展性，指面对需求变化与规模增长时，能以较小改动、较低风险完成修改与增强的能力。本实验在实现滑动窗口最大值的过程中，**已做一定扩展性设计**；同时也预留了在**数据、需求、用户、软件构建**等维度的后续扩展空间。本章结合具体代码简要说明。

8.1 已实现的扩展

(1) 算法实现可切换

- 主程序同时保留 **朴素算法**(`max_sliding_window_naive`)与 **单调队列优化**(`max_sliding_window`)；
- 通过统一入口 `solve(use_naive=False)` 切换实现，调用方无需关心内部细节；
- 原始代码另存为 `sliding_window_max_naive.py`，便于对比与单独测试。

新增第三种算法时，只需增加成员方法并在 `solve()` 中增加分支或改为策略映射，**不必改动**输入校验与测试框架。

(2) 数据输入与校验分离

- `set_array(nums, k)`：负责接收数组与窗口大小；
- `array_is_legal()`：独立负责合法性判断与错误提示；
- 求解函数只依赖 `self.nums`、`self.k`、`self.legal`，与数据来源解耦。

当前通过函数参数传入数据；后续若支持文件、标准输入或网络流，可增加 `set_array_from_file()` 等，最终仍调用 `set_array` 或写入 `self.nums`，**校验逻辑可复用**。

(3) 数据规模

- 使用 Python `list` 存储数组，长度理论上可任意扩展，已用 Profile 验证 $n = 500000$ 规模；
- 窗口 k 与数组长度独立配置，支持 $k = 1$ 至 $k = n$ 等边界（第五章已测）。

(4) 测试与工程辅助分离

函数	作用	扩展意义
<code>unit_test()</code>	单元测试入口	新增测试场景时统一调用
<code>profile_test()</code>	性能分析入口	与功能代码分离，便于换规模/算法
<code>data/test_cases.json</code>	外部测试数据	增删用例无需改测试代码核心逻辑

(5) 小结：已扩展功能一览

维度	已实现
算法	朴素 + 优化双实现， <code>solve()</code> 统一调度
数据	参数传入、 <code>list</code> 支持可变长、JSON 外部用例
校验	独立 <code>array_is_legal()</code> ，错误信息可扩展
工程	测试/性能/演示 (<code>main</code>) 分函数，模块文件拆分

8.2 未来可扩展方向

(1) 数据方面

方向	说明
数据来源	增加从文件、标准输入、网络接口、数据库读取数组的函数，统一汇入 <code>set_array</code>
数据类型	当前仅支持 <code>int</code> ；可扩展为 <code>float</code> 或自定义可比类型，在 <code>array_is_legal()</code> 中放宽类型检查
流式数据	对无限长数据流，可维护固定大小 <code>deque</code> ，每次新元素到达时 $O(1)$ 更新窗口最大值，无需一次性加载全数组

(2) 需求方面

方向	说明
返回值扩展	除最大值数组外，可同时返回每个最大值对应的窗口下标或窗口区间
统计扩展	在同一滑动窗口框架下扩展为求最小值、求第 k 大值等（需改队列单调性）
约束扩展	支持可变窗口长度、带权最大值等变体题目
算法自适应	当 k 很小时朴素法常数小，可通过 <code>solve()</code> 内根据 n 、 k 自动选型

(3) 用户方面

方向	说明
交互方式	main() 扩展为命令行参数 (argparse): 指定输入文件、 k 、是否使用朴素算法
结果呈现	除打印列表外, 输出到文件、JSON, 或简单可视化 (折线图展示最大值曲线)
配置化	用配置文件指定默认 k 、算法模式、日志级别, 适应不同使用场景

(4) 软件构建方面

方向	说明
模块化打包	将 src 整理为可安装包 (pip install -e .), 对外暴露稳定 API
持续集成	在 Git 仓库中配置 CI: 自动运行 pylint、unittest、coverage
插件式算法	用字典或注册机制管理多种 solve 实现 (策略模式)
文档与类型	补充 typing 协议、README、Sphinx 文档, 便于他人二次开发

8.3 小结: 作业问题 b) 一览

问题	回答
是否考虑可扩展性?	是; 类封装、算法可切换、校验与求解分离、测试数据外置等
已扩展了什么?	双算法、solve 统一入口、独立校验、JSON 测试数据、测试/性能辅助函数 (§8.1)
未来如何扩展?	数据 (多来源/流式)、需求 (返回值与题型变体)、用户 (CLI/可视化)、软件构建 (包管理/CI/插件) (§8.2)

9 两个原则

单一职责原则 (SRP) 与 **开放-封闭原则 (OCP)** 是面向对象设计中两条常用原则, 有助于降低耦合、提高可维护性与可扩展性。本实验在 `SlidingWindowMax` 类及相关模块的设计中遵循这两条原则。本章结合具体代码说明, 并指出**后续扩展时可复用、无需修改**的基础部分。

9.1 单一职责原则 (SRP)

9.1.1 原则含义

SRP 要求: 一个类或模块应只有一个引起它变化的原因, 即只承担一项职责。职责分离后可提高内聚、便于测试, 修改某一功能时影响范围更小。

9.1.2 本实验中的体现

`SlidingWindowMax` 类将不同关注点拆分到不同方法中:

方法	单一职责	变化原因
<code>set_array(nums, k)</code>	接收并保存输入数据	输入方式改变（如增加文件读入）
<code>array_is_legal()</code>	判断输入是否合法并提示	校验规则改变（如支持 float）
<code>max_sliding_window_naive()</code>	朴素算法求解	暴力算法实现细节调整
<code>max_sliding_window()</code>	单调队列算法求解	优化算法实现细节调整
<code>solve(use_naive=...)</code>	调度具体算法	新增算法入口或选择策略

类外部的辅助函数也职责分明：

函数	职责
<code>unit_test()</code>	为单元测试提供统一调用
<code>profile_test()</code>	为性能分析提供统一调用
<code>main()</code>	命令行演示

示例：若仅需修改「空数组」的提示文案，只需改 `array_is_legal()`，不必改动求解算法；若仅优化 deque 逻辑，只需改 `max_sliding_window()`，不必改动校验与测试代码。第五章白盒测试也按职责分别覆盖校验与求解分支，从侧面验证了职责划分合理。

9.1.3 独立原始模块

`sliding_window_max_naive.py` 中的 `SlidingWindowMaxNaive` 仅负责朴素求解，与主程序的校验、优化、测试辅助分离。

9.2 开放-封闭原则（OCP）

9.2.1 原则含义

OCP 要求：**软件实体应对扩展开放、对修改封闭**。即增加新功能时，尽量通过**新增代码**实现，而**少改已稳定、已测试通过的旧代码**。

9.2.2 本实验中的体现

（1）算法扩展

- 已有 `max_sliding_window_naive()` 与 `max_sliding_window()` 两种实现；
- 对外统一通过 `solve(use_naive=...)` 调用；
- 若将来增加第三种算法（如分块算法），可**新增** `max_sliding_window_xxx()` 并在 `solve()` 中增加分支或策略表，**原有两种算法的实现代码可保持不变**。

（2）输入与校验扩展

- 输入校验封装在 `array_is_legal()` 中，求解函数只检查 `self.legal`；

- 新增数据来源时，扩展 `set_array` 或增加新的 setter，**求解核心逻辑无需修改**。

(3) 测试与性能扩展

- 新增测试用例：改 `data/test_cases.json` 或增 test 方法，**被测求解代码不必改**；
- 新增性能场景：改 `profile_test()` 参数或脚本，**算法实现不必改**。

9.2.3 与第八章的关系

第八章所述「算法可切换、校验分离、JSON 外置测试数据」等设计，正是 OCP 在工程上的具体落地：**扩展点清晰，稳定代码封闭**。

9.3 可复用且稳定的基础功能

作业 c) 要求指出：**哪些部分是后续扩展时可重复使用、一般不需要修改的基础功能**。本实验认为以下部分属于该范畴：

基础部分	作用	扩展时是否需改
<code>array_is_legal()</code>	统一输入校验与错误提示	仅当校验规则变化时修改
<code>set_array()</code> 接口	统一的数据绑定入口	一般不改；新来源可新增函数调用它
<code>solve()</code> 调用约定	对外统一求解入口	一般不改；仅增加内部分支
<code>unit_test()</code>	测试框架入口	不改；测试用例在外部扩展
已验证的 <code>max_sliding_window()</code>	默认优化实现	题目不变则 不修改 ，作为稳定核心
<code>data/test_cases.json</code>	测试数据	增删用例即可，不改代码

会随扩展而新增、而非修改旧代码的部分：新算法方法、新输入函数、新测试方法、新配置文件等。

简要结论：校验层 (`array_is_legal`)、调度层 (`solve`)、测试辅助层 (`unit_test`) 构成**稳定基础**；算法层通过**新增方法**扩展，符合 OCP。

9.4 小结：作业问题 c) 一览

问题	回答
是否遵守 SRP?	是；输入、校验、各算法、调度、测试/性能辅助职责分离 (§9.1)
是否遵守 OCP?	是；新算法、新数据源、新测试以 扩展 为主，少改稳定核心 (§9.2)
哪些是可复用、不常改的基础功能?	<code>array_is_legal</code> 、 <code>set_array</code> 接口、 <code>solve</code> 约定、 <code>unit_test</code> 、稳定优化算法及外部测试数据 (§9.3)

10 错误处理

健壮程序应能识别**非法输入**并给出明确反馈，避免 silently 产生错误结果或直接崩溃。本实验采用「**事前校验 + 错误提示 + 安全返回**」的方式处理错误：在 `array_is_legal()` 中集中判断输入

合法性，不合法时打印中文提示、置 `legal=False`，求解函数返回 `None` 而不执行核心算法。

10.1 输入约定（前置条件）

本程序约定合法输入须同时满足：

条件	说明
nums 为 list	不接受 tuple、str 等
$\text{len}(\text{nums}) > 0$	数组非空
k 为 int	不接受 float 等
$k > 0$	窗口大小为正整数
$k \leq \text{len}(\text{nums})$	窗口不能超出数组
nums 中每个元素为 int	本实验不支持 float

以上规则在 `array_is_legal()` 中逐项检查。

10.2 已考虑的错误情形

序号	错误情形	检测方式	用户提示（摘要）	程序行为
1	nums 非 list	<code>isinstance(self.nums, list)</code>	「输入必须是列表类型」	<code>legal=False</code>
2	空数组	<code>len(self.nums) == 0</code>	「数组不能为空」	<code>legal=False</code>
3	k 非 int	<code>isinstance(self.k, int)</code>	「k 必须是整数」	<code>legal=False</code>
4	$k \leq 0$	<code>self.k <= 0</code>	「k 必须为正整数」	<code>legal=False</code>
5	$k > \text{len}(\text{nums})$	长度比较	「k 不能大于数组长度」	<code>legal=False</code>
6	元素非 int	遍历 <code>isinstance(value, int)</code>	「数组元素必须是整数」	<code>legal=False</code>
7	非法时仍调用求解	<code>solve()</code> / <code>max_sliding_window*()</code> 内判断 <code>self.legal</code>	「输入不合法，无法求解」	返回 <code>None</code>

原始模块 `SlidingWindowMaxNaive` 对 $k \leq 0$ 或 $k > n$ 直接返回 `None`（无打印），校验能力弱于主程序；主程序通过 `array_is_legal()` 提供完整提示。

10.3 处理机制说明

(1) 校验与求解分离

校验逻辑集中在 `array_is_legal()`，求解函数只依赖 `self.legal`，避免在多处重复判断。

(2) 错误信息

- 采用 `print` 输出中文提示，面向实验/演示场景，便于用户直接看到原因；
- 各分支提示文案不同，便于定位是哪一种非法输入（第五章测试用 `assertIn` 验证关键词）。

(3) 失败时的返回值

- 非法输入下 `solve()`、`max_sliding_window()`、`max_sliding_window_naive()` 均返回 `None`，不抛异常、不返回部分错误结果，避免调用方误用；
- 调用方可通过 `if result is None` 判断是否求解成功。

(4) 测试保障

第五章 `TestSlidingWindowValidation` 对序号 1~7 对应场景编写白盒测试，使用 `patch(sys.stdout)` 捕获提示并断言 `legal` 与返回值，保证错误处理行为可回归验证。

对本实验范围（内存中的 `list` + `int` 输入）而言，**条件校验 + 明确提示 + 返回 `None`** 已满足要求。

10.4 小结：作业问题 d) 一览

问题	回答
考虑了哪些错误？	类型错误、空数组、非法 k 、非 <code>int</code> 元素、非法时仍求解等 (§10.2 共 7 类)
如何处理？	<code>array_is_legal()</code> 集中校验 → 打印提示 → <code>legal=False</code> → 求解返回 <code>None</code>
如何验证？	第五章 9 个白盒测试 + <code>stdout</code> 断言

11 完整程序代码

11.1 源代码文件清单

11.1.1 核心源代码

文件	说明
<code>src/sliding_window_max.py</code>	主程序：校验、朴素/优化双算法、 <code>solve</code> 调度、测试与性能辅助
<code>src/sliding_window_max_naive.py</code>	原始代码（第二章）：独立朴素实现，作对比基线
<code>src/test_sliding_window_max.py</code>	单元测试（第五、六章）：20 个 <code>test</code> 方法
<code>data/test_cases.json</code>	测试数据：6 组标准用例（含题目示例）

11.1.2 辅助脚本与配置

文件	说明
src/.pylintrc	PyLint 配置（第三章）
src/.coveragerc	Coverage 配置（第六章）
src/run_pylint.py	生成 PyLint 详细报告（-ry）
src/benchmark_time.py	墙钟时间对比（第四章）
src/run_profile_naive_only.py	朴素算法 Profile
src/run_profile_optimized_only.py	优化算法 Profile
src/run_profile.py	一键 Profile 分析
src/run_coverage.py	一键 unittest + coverage
src/before_pylint/	PyLint 修复前代码

11.2 实验复现命令汇总

在项目根目录下执行：

```

1  # 运行题目示例演示
2  python src/sliding_window_max.py
3
4  # Pylint (修复后)
5  pylint --rcfile=src/.pylintrc src/sliding_window_max.py
   src/sliding_window_max_naive.py
6  python src/run_pylint.py
7
8  # Pylint (修复前代码)
9  pylint src/before_pylint/sliding_window_max.py
   src/before_pylint/sliding_window_max_naive.py
10
11 # Profile 性能分析
12 python src/benchmark_time.py
13 python src/run_profile_naive_only.py
14 python src/run_profile_optimized_only.py
15
16 # Unittest
17 python src/test_sliding_window_max.py
18
19 # Coverage
20 python src/run_coverage.py
21 # 浏览器打开 my_coverage_result/index.html

```

12 github 仓库

代码详见：<https://github.com/Wangxuuuuu/Software-Engineering>